

Geo-Aware WebAssembly (WASM) Serverless Orchestrator

Cao Trần Khánh Linh - 23BI14252

University of Science and Technology of Hanoi
Department of Information and Communication Technology

External Supervisor: Eng. Trần Tuấn Anh Internal Supervisor: Assoc. Prof. Trần Giang Sơn

July 13, 2026

Outline

- ① Introduction
- ② Architectural Design
- ③ Execution Lifecycle
- ④ Results
- ⑤ Conclusion

Edge computing

- Compute close to the data / user
- Latency-sensitive workloads
- Where matters, not only how much

Serverless

- Submit prebuilt functions, not servers
- Platform picks node, runs, returns
- Developers never pick worker manually

wasmCat: Native Go master-worker orchestrator running WebAssembly functions on geo-distributed edge workers with no container runtime.

Why WebAssembly?

Container overhead

- Image pull + layer unpack
- Namespace / cgroup setup
- Heavy control plane
- Scheduling ignores geography

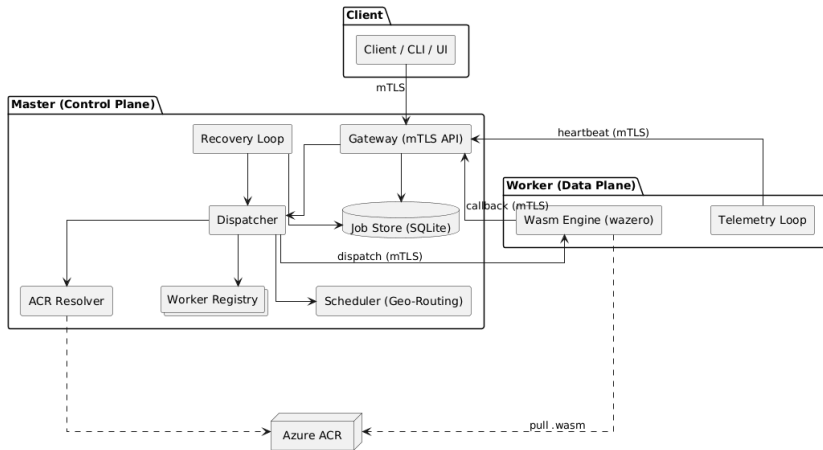
WebAssembly + wazero

- One `.wasm`, any host
- Sandbox: linear memory only
- Compile once, instantiate per request
- Pure-Go runtime, embedded

=> Edge functions need fast startup + minimal dependencies + location-aware placement.

- ① Go-native master-worker orchestrator
- ② Capacity-aware geo-routing with Haversine formula
- ③ Embedded wazero: 2 ABIs, digest-aware cache, strict limits
- ④ Security: mTLS, certificate-bound identity, digest verification
- ⑤ Reliability: durable SQLite jobs, idempotency
- ⑥ Tooling: CLI, web dashboard, JIT provisioning from Azure ACR

System Architecture



Data Model: Input → Output

INPUT (**ExecutionRequest**)

```
struct {  
    RequestID      string  
    ModuleName     string  
    Payload        string  
    ModuleRegistryURL string  
    ModuleDigest   string  
    UserLat        float64  
    UserLon        float64  
}
```

OUTPUT (**ExecutionResponse**)

```
struct {  
    RequestID      string  
    Result         string  
    ExecutionTimeMs float64  
    ExecutedOnNodeID string  
    Error          string  
}
```

POST /api/v1/execute ⇒ JSON in → JSON out (same request_id)

Worker Model & Registry

One worker = one **WorkerNode**

```
struct {  
    ID          string  
    IPAddress   string  
    Latitude    float64  
    Longitude   float64  
    CPUFree     float64  
    RAMFreeMB   float64  
    LastSeen    time.Time  
    State       string  
}
```

Registry = **map[ID]WorkerNode**

- In memory, behind a `sync.RWMutex`: many reads, rare writes
- Heartbeat every 5s updates only CPU / RAM
- Coordinates set once, at registration
- Background loop evicts workers that go silent

Two stages

- ① Filter: drop draining, then below CPU/RAM thresholds
- ② Score: nearest by great-circle distance

Input: `user_lat/lon` + workers

Output: one `WorkerNode`

Haversine ($R = 6371$ km)

$$a = \sin^2 \frac{\Delta \text{lat}}{2} + \cos \text{lat}_1 \cos \text{lat}_2 \sin^2 \frac{\Delta \text{lon}}{2}$$

$$d = R \cdot 2 \operatorname{atan2}(\sqrt{a}, \sqrt{1-a})$$

Shorter distance usually lowers latency, but the network path and congestion also matter.

Execution Modes

wasmCat ABI (custom)

```
// module exports: memory, malloc, run
run(ptr uint32, len uint32) uint64
// in : payload bytes at [ptr, ptr+len)
// out: packed uint64
outPtr = uint32(packed >> 32)
outLen = uint32(packed & 0xFFFFFFFF)
```

WASI (wasip1)

```
// module exports: _start
stdin  <- payload
_start()           // exit 0 = ok
stdout -> result
```

Auto-detect (abi = "" | wasmcat | wasi)

exports run $\Rightarrow$ custom | exports _start $\Rightarrow$ WASI. Both: same digest cache, fresh instance per request.

- ① mTLS: Verified client cert before any handler
- ② Certificate-bound identity: Claimed worker ID must equal cert CN / DNS SAN $\Rightarrow$ no impersonation
- ③ Module integrity: Verified SHA-256 hash before compile; digest is part of cache key
- ④ Scoped credentials: ACR token: repository-scoped, pull-only, short-lived

Synchronous Execution: Input → Process → Output

Pipeline

- 1 Validate + idempotency (request_id)
- 2 Resolve ACR (if needed)
- 3 Scheduler → nearest worker
- 4 /invoke over mTLS
- 5 Worker: fetch/compile, run, read output

Input

```
{ "request_id":"req_demo_001",  
  "module_name":"jsonfmt",  
  "payload":{"\"b\":2,\"a\":1}",  
  "user_lat":1.29, "user_lon":103.85 }
```

Output

```
{ "request_id":"req_demo_001",  
  "result":{"\n \"a\":1,\n \"b\":2\n}",  
  "execution_time_ms":2.932,  
  "executed_on_node_id":"worker-vn-01" }
```

Durable Job Process & State Machine

- `POST /api/v1/jobs` → 202, persisted in SQLite
- Recovery loop leases + dispatches queued jobs
- Idempotency: same fingerprint → replay; different → 409

ambiguous

Lease expired, execution unconfirmed $\Rightarrow$ mark ambiguous, never blind-retry.
Correctness over convenience.

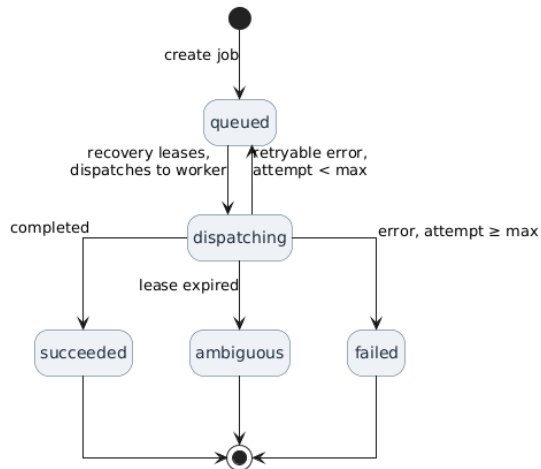


Figure: Job lifecycle

JIT Module Provisioning from Azure ACR

- ① Detect *.azurecr.io reference
- ② Master: DefaultAzureCredential → token repository:<name>:pull
- ③ Fetch OCI manifest → pick application/wasm layer → blob URL
- ④ Forward {blob URL + bearer + digest} to worker → download & compile

Property

Workers hold no Azure credentials, only a per-request, repository-scoped, pull-only token. Modules published as OCI artifacts with ORAS.

Result 1: Geo-routing

Client coords	Location	Selected worker
1.29, 103.85	Singapore	worker-vn-01 (SEA)
47.67, -122.12	Seattle	worker-west-us-01 (US)

Table: Same request, different coordinates $\rightarrow$ nearest eligible worker.

- Draining: drained worker excluded; traffic shifts to the ready one
- mTLS: plain `curl` (no client cert) rejected at transport layer

Result 2: Latency

Module source	Cold (ms)	Warm (ms)
Direct HTTP URL	2135.559	7.529
Azure ACR (OCI)	2372.821	2.932

Table: Worker `execution_time_ms` by source and cache state.

- Cold $\approx$ download + wazero compile: paid once per version per worker
- Warm $\approx$ fresh instance from cache: a few ms
- No image pull / namespace setup on any request

- ① Single master: local SQLite recovery, not HA
- ② In-memory registry: membership rebuilt from heartbeats after restart
- ③ Haversine $\neq$ RTT: no congestion / cache-locality awareness
- ④ Coarse isolation: host-level CPU/RAM, no per-invocation cgroup / fuel
- ⑤ Workload: stateless functions only (no sockets / FS beyond `wasip1`)

Demonstrated

A compact native Go system: capacity- & location-aware scheduling, JIT ACR provisioning, mTLS, durable idempotent jobs on a two-region Azure VM deployment.

Future work

- High Availability control plane: leader election + external job store
- Smarter scheduling: RTT, queue depth, cache locality
- Stronger isolation: fuel metering, cgroups, admission control
- Wider model: HTTP-handler ABI, WASI-HTTP, component model

Thank you for listening